

DecodeLabs AI Project 1

LogicBot - Rule-Based AI Chatbot

Professional Project Documentation

Internship Track	Artificial Intelligence
Training Kit	Industrial Training Kit - Batch 2026
Project Type	Rule-Based AI Chatbot
Language	Python 3
Developer	Syed Muneeb Haider
Submission Status	Ready for Review

This document explains the problem statement, design approach, implementation flow, features, testing, and final submission evidence for the Project 1 chatbot.

Table of Contents

- 1. Executive Summary
- 2. Project Objective and Scope
- 3. Requirement Mapping
- 4. System Architecture
- 5. Implementation Details
- 6. Rule-Based Knowledge Base
- 7. Humanized Improvements
- 8. Testing and Validation
- 9. How to Run the Project
- 10. Sample Output
- 11. Limitations and Future Enhancements
- 12. Conclusion

1. Executive Summary

LogicBot is a professional Python-based rule-based chatbot created for DecodeLabs Artificial Intelligence Project 1. The project focuses on deterministic control flow rather than machine learning. Every answer is selected from clear, visible rules, making the chatbot easy to inspect, debug and explain during internship verification.

The final version goes beyond the minimum requirement by adding a clean folder structure, exact intent matching, keyword-based matching, input sanitization, fallback responses, session memory, traceability, chat logging, sample output and unit tests. The chatbot is still fully rule-based and remains aligned with the foundation-level learning goal.

Core Idea

A chatbot that responds to predefined questions using if-else logic and dictionaries.

Strength

69 exact question patterns and 27 keyword rule groups.

Verification

Unit tests passed: 10 tests completed successfully.

2. Project Objective and Scope

The project objective is to create a simple chatbot that responds to predefined user inputs. It must handle greetings, recognize exit commands, use if-else logic and run continuously until the user chooses to stop the program.

The implementation scope includes core chatbot behavior, clean input processing, rule matching, response generation, a fallback strategy for unknown questions, and a short documentation package so the project can be reviewed quickly by the company.

Project Scope

- Accept user input through the terminal.
- Clean the input by converting it to lowercase, removing punctuation and trimming spaces.
- Use if-else logic for empty input, exit commands, memory commands, utility commands and fallback handling.
- Use dictionaries as a clean knowledge base for exact phrase matching and response selection.
- Use keyword groups to answer natural variations of the same question.
- Provide useful answers about AI, DecodeLabs, Project 1, certificates, submission and rule-based systems.

3. Requirement Mapping

The following table maps the official Project 1 expectations to the completed implementation. This makes verification easy and shows that the work is not only functional but also well organized.

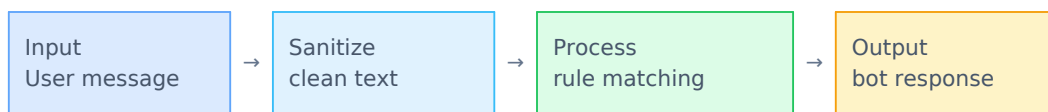
Requirement	Status	Implementation
-------------	--------	----------------

Handle greetings	Completed	DIRECT_INTENTS, KEYWORD_INTENTS and RESPONSES
Handle exit commands	Completed	EXIT_COMMANDS with clean break from the loop
Use if-else logic	Completed	generate_response() and detect_intent() decision flow
Continuous loop	Completed	while True inside run_chatbot()
Predefined responses	Completed	RESPONSES dictionary
5+ intents	Exceeded	69 exact inputs and 27 keyword rule groups
Fallback response	Completed	Fallback replies for unknown inputs
Input sanitization	Completed	sanitize_input() handles case, punctuation and spaces
Clean project structure	Completed	Separate src, docs, tests and assets folders

The project kit describes Project 1 as a Rule-Based AI Chatbot and highlights greetings, exit commands, if-else logic and a continuous loop as key requirements. This implementation directly covers those areas and adds professional enhancements while staying inside the rule-based scope.

4. System Architecture

LogicBot follows the IPO model: Input, Process and Output. The user enters text, the bot sanitizes it, the processing layer checks rules, and the selected response is printed in the terminal.



Architecture Flow

```
Raw Input
-> Input Sanitization
-> Empty / Exit / Memory / Utility Checks
-> Exact Dictionary Match
-> Keyword Intent Match
-> Fallback Response
-> Output
```

This structure creates a white-box chatbot because the path from user input to bot response can be explained. The trace command strengthens this design by showing whether the answer came from an exact dictionary match, keyword match or fallback rule.

5. Implementation Details

5.1 Folder Structure

The project is separated into source code, tests, documentation and assets. This makes the submission cleaner than a single-file project and helps reviewers inspect the work faster.

```
decelabs_ai_project1_rule_based_chatbot/
|
|-- main.py
|-- requirements.txt
|-- run_chatbot.bat
```

```
| -- run_chatbot.sh
| -- README.md
|
| -- src/
|   |-- chatbot.py
|   |-- intents.py
|   |-- utils.py
|
| -- tests/
|   |-- test_chatbot.py
|
| -- docs/
|   |-- project_report.md
|   |-- question_bank.md
|   |-- submission_checklist.md
|   |-- project_documentation.pdf
|
| -- assets/
|   |-- sample_output.txt
```

5.2 Main Files

File	Purpose
main.py	Entry point for running the chatbot from the terminal.
src/chatbot.py	Contains the chatbot class, continuous loop, response generation, trace and stats behavior.
src/intents.py	Stores direct intents, keyword intent groups, exit commands and response templates.
src/utils.py	Stores input sanitization, name extraction, response selection, time/date and logging helpers.
tests/test_chatbot.py	Contains automated unit tests for the most important chatbot behavior.
assets/sample_output.txt	Contains a ready demo conversation for screenshots and verification.

5.3 Core Logic Snippet

The chatbot uses a readable decision order. Important checks happen first, then the bot attempts exact matching, keyword matching and fallback response generation.

```
if clean_input == "":
    return empty_input_response

if clean_input in EXIT_COMMANDS:
    return goodbye_response, True

if detected_name:
    save_name_for_this_session()

if clean_input in {"trace", "why this answer"}:
    return trace_message

intent, method = detect_intent(clean_input)
reply = choose_response(RESPONSES.get(intent, RESPONSES["fallback"]))
```

6. Rule-Based Knowledge Base

The knowledge base is stored in dictionaries. This keeps the project easy to expand and avoids a very long if-elif ladder. Exact user inputs are mapped to intents, and each intent has one or more

response templates.

Intent Coverage

Category	Examples
Basic Chat	hello, hi, salam, thank you, ok, bye
AI Concepts	what is AI, chatbot, rule-based AI, guardrails, white-box AI, IPO model
DecodeLabs	what is DecodeLabs, what is the internship, Project 1, requirements, skills, contact
Fee Safety	certificate fee, is certificate free, is internship free, is it paid
Submission Quality	how to submit, submission tips, how to get recommendation
Memory and Utility	my name is Muneeb, what is my name, trace, stats, time/date

Why Dictionary Lookup Is Better Here

A large if-elif ladder is harder to maintain as the question bank grows. A dictionary makes the code easier to read because a phrase can be connected directly to an intent. Adding a new question usually means adding one new key-value pair instead of editing the whole decision structure.

7. Humanized Improvements

The project was improved to feel less robotic while still remaining rule-based. The responses are written in a clear, friendly and practical tone. Several response variations are stored for common intents so the bot does not repeat the exact same sentence every time.

- Human-like answers for Project 1, DecodeLabs, certificate fee, submission and recommendation questions.
- Safe answers for uncertain fee or certificate details instead of unsupported claims.
- Session memory so the bot can store and recall the user name during the current run.
- Trace command to explain the previous response path and demonstrate white-box behavior.
- Stats command to show total exact inputs, keyword groups and session message count.
- Cleaner source code with minimal comments and readable function names.

8. Testing and Validation

The project includes automated unit tests to verify important behavior. Tests help prove that the chatbot does not only work during a manual demo but also passes repeatable checks.

Command:

```
python -m unittest discover tests
```

Result:

```
.....
```

```
-----  
Ran 10 tests in 0.001s
```

OK

Tested Areas

- Input sanitization and punctuation cleanup.
- Name extraction and session memory.
- Greeting response behavior.
- Project 1 and DecodeLabs answers.
- Certificate fee safety answer.
- Exit command handling.
- Trace response and fallback response.

9. How to Run the Project

Windows

Open the project folder in Command Prompt and run the following command:

```
python main.py
```

Or double-click the Windows helper file:

```
run_chatbot.bat
```

Linux / macOS

```
python3 main.py
```

```
# Optional helper script:  
chmod +x run_chatbot.sh  
./run_chatbot.sh
```

No external Python packages are required for the chatbot itself. The requirements file is kept simple because the project uses only the Python standard library.

10. Sample Output

The following conversation can be used for project screenshots and verification. It demonstrates greetings, Project 1 explanation, requirements, DecodeLabs questions, fee-safety handling, memory, trace, stats and clean exit.

```
LogicBot | DecodeLabs AI Project 1 | Rule-Based AI Chatbot  
Ask about AI, Project 1, DecodeLabs, requirements, certificate fee, or submission tips.  
Type 'help' for examples. Type 'exit' or 'bye' to close.
```

```
You: hello  
LogicBot: Hello! I am LogicBot. I can help you understand AI Project 1, DecodeLabs  
questions, and rule-based chatbot concepts.
```

```
You: what is project 1  
LogicBot: Project 1 is a Rule-Based AI Chatbot. It is a foundation task where the bot  
responds to predefined inputs using control flow and logic.
```

```
You: requirements  
LogicBot: The core requirements are: handle greetings, handle exit commands, use if-  
else logic, and run in a continuous loop.
```

```
You: what is decode labs
```

LogicBot: DecodeLabs is shown in this training kit as the organization powering the Batch 2026 Artificial Intelligence Project 1 industrial training task.

You: certificate fee

LogicBot: I should not guess about certificate payment. This project kit does not clearly say whether the certificate has a separate fee, so the safest step is to confirm through DecodeLabs official contact.

You: my name is Muneeb

LogicBot: Nice to meet you, Muneeb. I will remember your name during this session.

You: what is my name

LogicBot: Your name is Muneeb. I stored it with a simple session variable.

You: trace

LogicBot: Last response came from: session memory check. That is why this bot is easy to inspect.

You: stats

LogicBot: Session stats: 8 user messages, 69 exact inputs, 27 keyword groups, saved name: Muneeb.

You: exit

LogicBot: Goodbye! Keep your code clean, your logic clear, and your submission professional.

11. Limitations and Future Enhancements

This chatbot is not a machine-learning model and does not understand every possible sentence. Its answers depend on predefined rules, direct inputs and keyword groups. This limitation is expected because the project is designed to teach rule-based AI foundations.

Future Enhancements

- Add a GUI interface using Tkinter or a simple web version using Flask.
- Move the knowledge base to JSON so new intents can be added without editing Python code.
- Add Urdu/Hindi response support for multilingual users.
- Add voice input and text-to-speech output.
- Add an admin panel to manage questions and responses.
- Add an optional LLM fallback after rule matching for a hybrid architecture.

12. Conclusion

LogicBot successfully completes the DecodeLabs AI Project 1 requirement and adds a professional layer of quality. It demonstrates control flow, decision-making logic, dictionaries, sanitization, fallback handling, session memory, traceability and testing. The final project is clean, explainable, easy to run and ready for internship submission.

Final Recommendation: Submit the ZIP file, this PDF documentation, screenshots of the running chatbot, and the unit test result screenshot for a stronger review impression.